Lab 5: Static Code Analysis

Python Static Analysis - Conference Room Scheduler | Lab 5 | Student | 2026-10-16

Aim

To use Pylint, Bandit, and Flake8 to identify and fix quality, security, and style problems in the conference room scheduler, then verify that the corrected program still works.

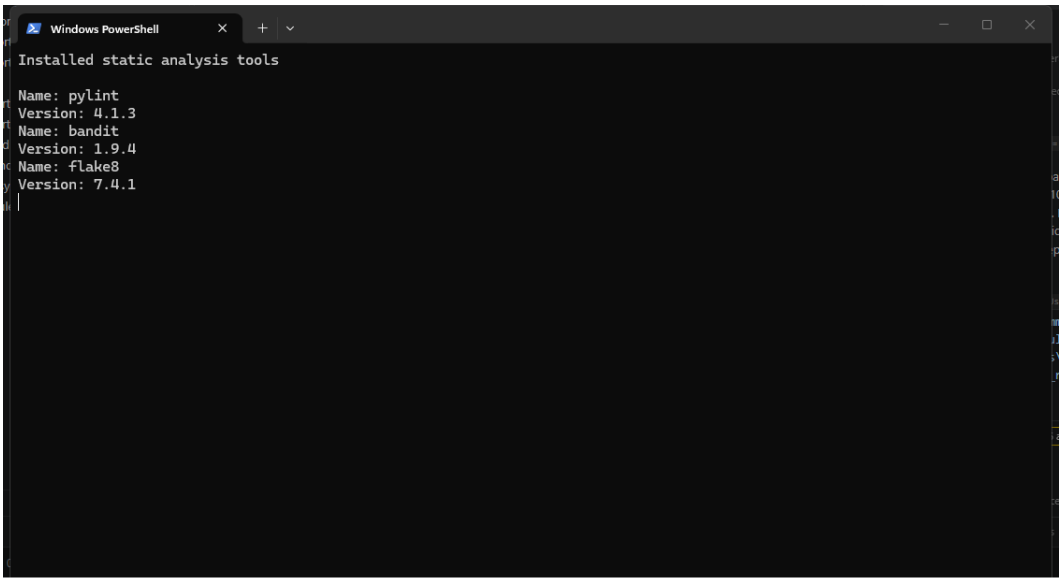
Procedure

I ran the original program on Windows with its default input, installed and ran all three analysis tools, documented the findings, corrected the scheduler, and checked behavior with unit tests and sample runs. The supplied three-page PDF requests four reflection answers but does not include four explicit question statements, so the reflection addresses four themes from its objectives and learning outcomes.

Task 1: Install and verify the analysis tools

Command(s) used:

```
python -m pip install pylint bandit flake8
python -m pip show pylint bandit flake8
```

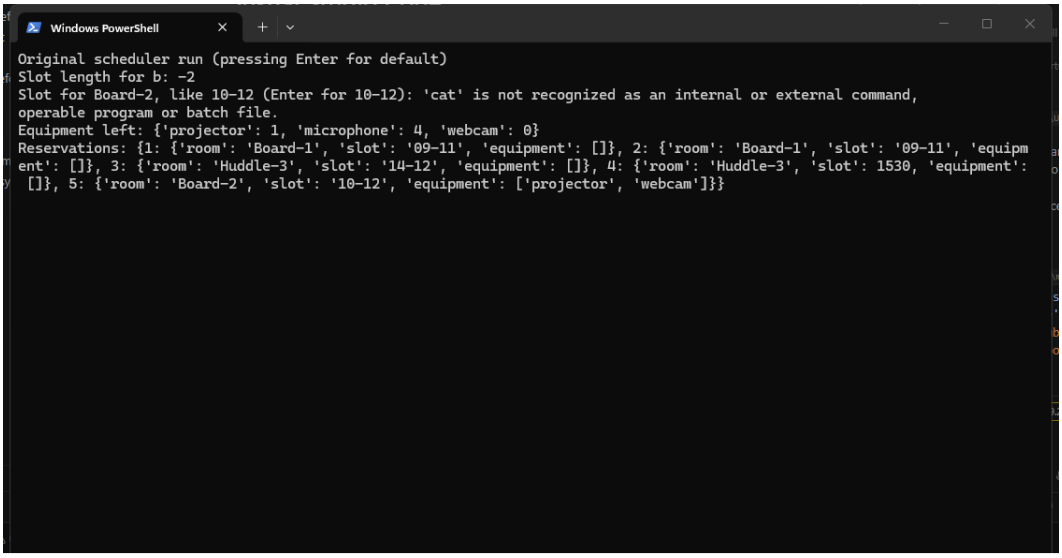


Observation: I installed the three requested tools and checked their versions in the terminal. Pylint 4.1.3, Bandit 1.9.4, and Flake8 7.4.1 were available for the analysis.

Task 2: Run the original scheduler

Command(s) used:

```
python scheduler_system.py (press Enter for the default slot)
```

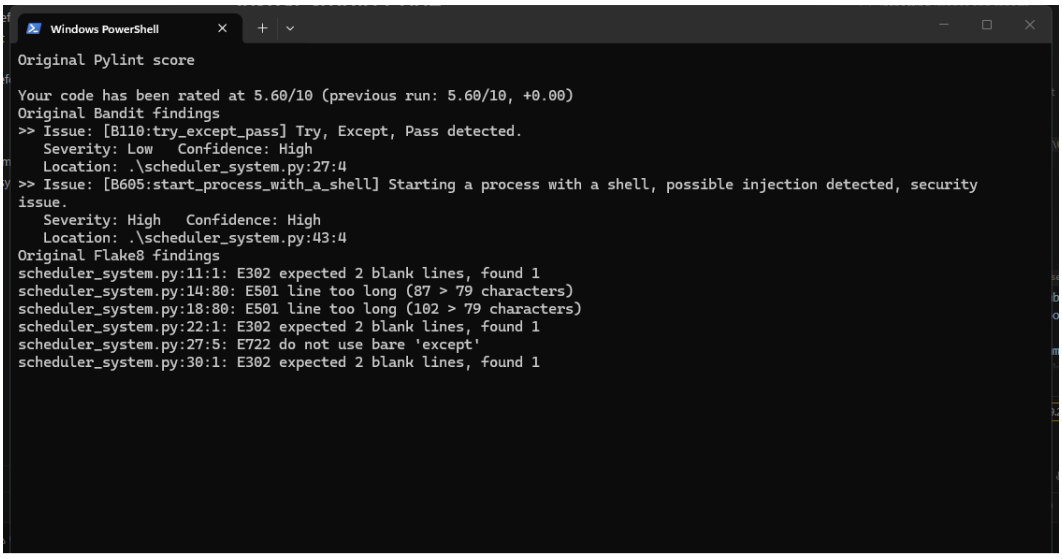


Observation: The initial run reported a slot length of -2 for the invalid 14-12 booking and accepted two bookings for Board-1 at the same time. It also tried to run the Unix-only cat command on Windows, which produced an error, while the integer time slot was stored without validation.

Task 3: Analyze and save the original findings

Command(s) used:

```
pylint scheduler_system.py
bandit -r scheduler_system.py
flake8 scheduler_system.py
```

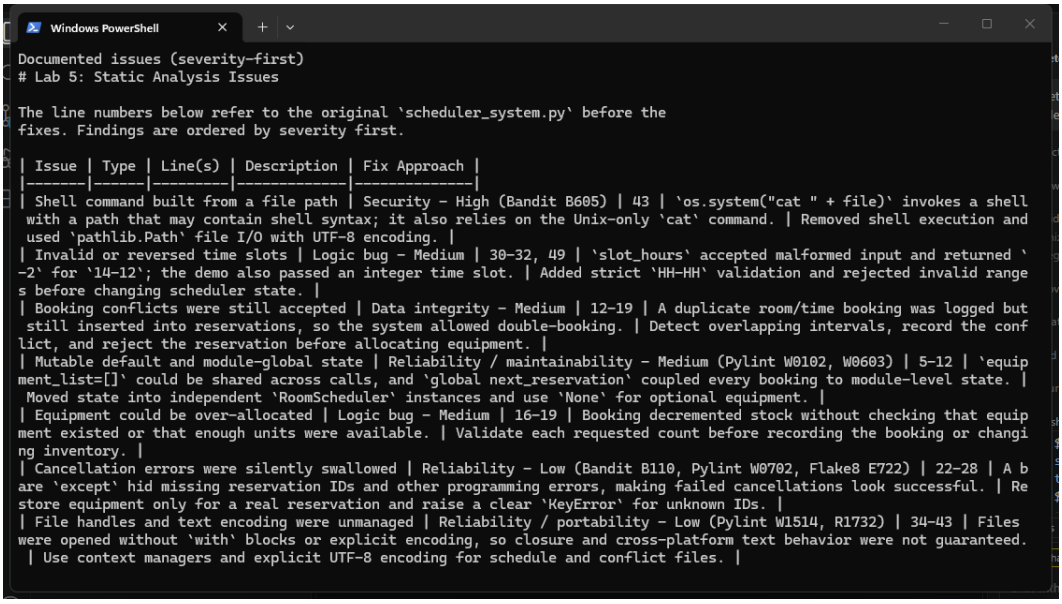


Observation: The original file scored 5.60/10 in Pylint. Bandit reported a high-severity shell invocation and a low-severity swallowed exception, while Flake8 reported formatting, exception, and unused-variable findings; the full before-fix output is saved in the repository.

Task 4: Document the issues

Command(s) used:

```
Get-Content issues.md
```

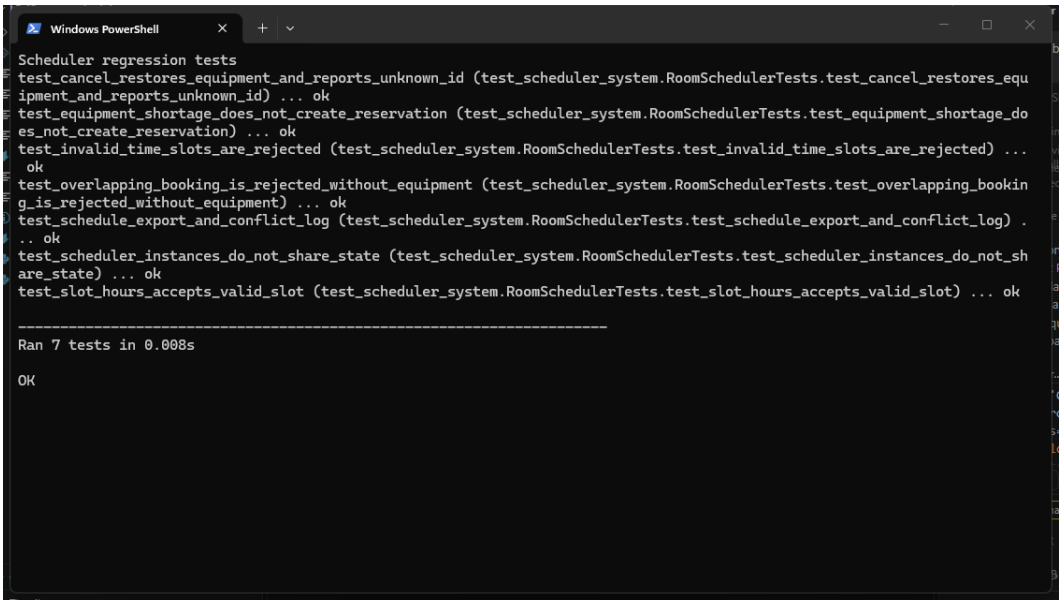


Observation: I recorded eight issues in severity-first order, including the Bandit shell warning, invalid time handling, double-booking, equipment over-allocation, and the analyzer-reported code-quality problems. Each row includes the original line numbers and a corresponding fix approach.

Task 5: Fix the scheduler and run regression tests

Command(s) used:

```
python -m unittest -v test_scheduler_system
```

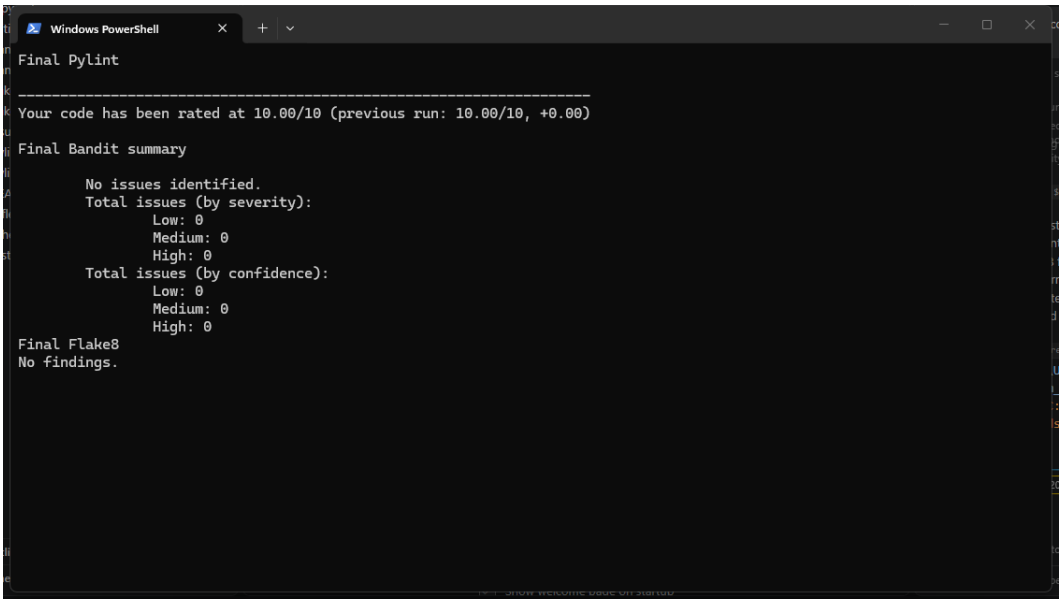


Observation: All seven regression tests passed. They cover malformed and reversed time slots, overlapping reservations, equipment availability and restoration, independent scheduler state, and JSON export plus conflict logging.

Task 6: Rerun all static analysis tools

Command(s) used:

```
pylint scheduler_system.py
bandit -r scheduler_system.py
flake8 scheduler_system.py test_scheduler_system.py
```

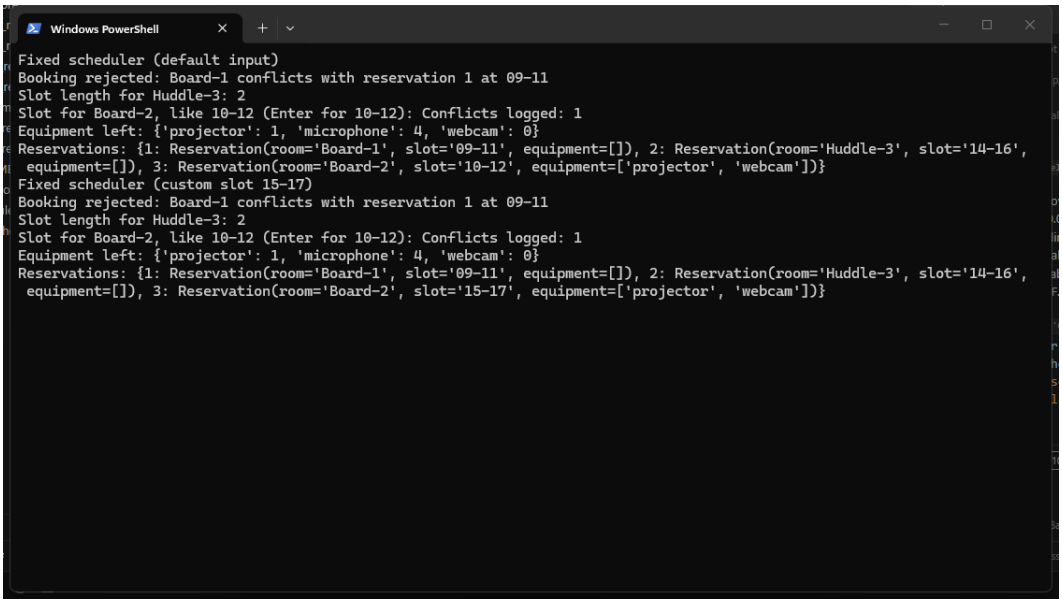


Observation: The final Pylint score is 10.00/10. Bandit reported no issues, and Flake8 reported no findings in either the scheduler or its tests; the final reports are saved as pylint_report.txt, bandit_report.txt, and flake8_report.txt.

Task 7: Run the corrected scheduler with sample inputs

Command(s) used:

```
python scheduler_system.py (press Enter for the default slot)
python scheduler_system.py (enter 15-17)
```

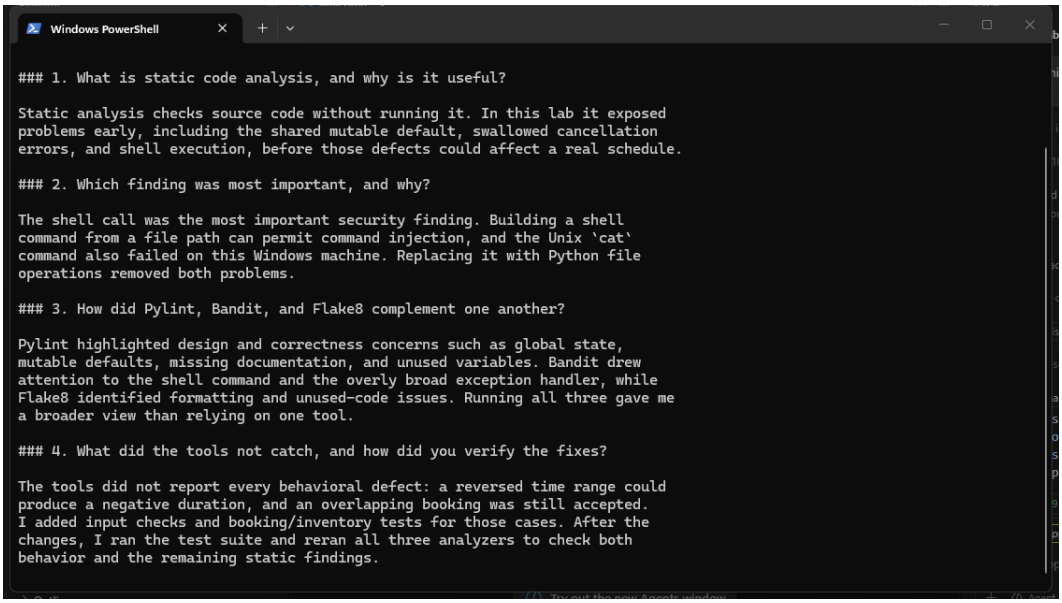


Observation: The corrected program reports the duplicate Board-1 booking as rejected and calculates the valid Huddle-3 slot as two hours. Both the default Board-2 slot and the custom 15-17 slot run successfully, and the equipment counts reflect the accepted reservations.

Task 8: Complete the reflection

Command(s) used:

```
Get-Content reflection.md
```



Observation: I reflected on why static analysis is useful, which security issue mattered most, how the three tools complement one another, and what required additional behavioral tests. The handout does not print the four questions themselves, so I clearly identified these as reflection themes rather than quoting prompts that were not provided.

Conclusion

The scheduler now rejects invalid time slots, overlapping room bookings, and equipment requests that exceed inventory. Its cancellation and file handling are explicit and portable. The seven regression tests passed, and the final Pylint, Bandit, and Flake8 reports are clean.

Appendix: Static Analysis Issue Table

The following findings refer to the original scheduler before fixes and are ordered by severity.

Issue	Type	Line(s)	Description	Fix Approach
Shell command built from a file path	Security - High (Bandit B605)	43	`os.system("cat " + file)` invokes a shell with a path that may contain shell syntax; it also relies on the Unix-only `cat` command.	Removed shell execution and used `pathlib.Path` file I/O with UTF-8 encoding.
Invalid or reversed time slots	Logic bug - Medium	30-32, 49	`slot_hours` accepted malformed input and returned `-2` for `14-12`; the demo also passed an integer time slot.	Added strict `HH-HH` validation and rejected invalid ranges before changing scheduler state.
Booking conflicts were still accepted	Data integrity - Medium	12-19	A duplicate room/time booking was logged but still inserted into reservations, so the system allowed double-booking.	Detect overlapping intervals, record the conflict, and reject the reservation before allocating equipment.
Mutable default and module-global state	Reliability / maintainability - Medium (Pylint W0102, W0603)	5-12	`equipment_list=[]` could be shared across calls, and `global next_reservation` coupled every booking to module-level state.	Moved state into independent `RoomScheduler` instances and use `None` for optional equipment.
Equipment could be over-allocated	Logic bug - Medium	16-19	Booking decremented stock without checking that equipment existed or that enough units were available.	Validate each requested count before recording the booking or changing inventory.
Cancellation errors were silently swallowed	Reliability - Low (Bandit B110, Pylint W0702, Flake8 E722)	22-28	A bare `except` hid missing reservation IDs and other programming errors, making failed cancellations look successful.	Restore equipment only for a real reservation and raise a clear `KeyError` for unknown IDs.
File handles and text encoding were unmanaged	Reliability / portability - Low (Pylint W1514, R1732)	34-43	Files were opened without `with` blocks or explicit encoding, so closure and cross-platform text behavior were not guaranteed.	Use context managers and explicit UTF-8 encoding for schedule and conflict files.
Style and unused variables	Style / maintainability - Low (Flake8 E302, E305, E501, F841)	11-59	Missing blank lines, overlong lines, and unused demo variables obscured the actual behavior.	Reformatted the module and removed unused assignments; verified with Flake8.